Credit Card Fraud Detection

Group Project Report

Group Leader: Jia Zhenzhen (1397278) `zhenzhenjia@ln.hk`
Members: Zhu Zhongxi (1397345), `zhuzhongxizhuzhongxi@LN.hk`; ZHENG Xinzhe (1394977), `xinzhezheng@ln.hk`; Hou Shuwei (1397266), `shuweihou@ln.hk`; Liu Zirui (1397503), `ziruiliu3@ln.hk`; Wang Wenxuan (1397228), `wenxuanwang3@ln.hk`

Due: 24 March 2026

Repository: https://github.com/houshuwei/Group-Assignment-Insight-Syndicate

---

Contribution Statement (Mandatory) [TODO:]

- **Leader:** Selected the project topic, assigned tasks to group members, and consolidated and verified all materials for the final submission.
- **Wang Wenxuan:** Selected the dataset and prepared the dataset description and problem background, including references.
- **Zhu Zhongxi:** Completed model training, validation, and generated simulation results for the chosen topic.
- **Liu Zirui:** Visualized the results produced in Task 3 and conducted result interpretation and analysis.
- **Hou Shuwei:** Organized the codebase, compiled performance metrics and figures, uploaded all materials to GitHub, and prepared the GitHub repository and README documentation.
- **ZHENG Xinzhe:** Compiled the topic introduction, background information, dataset description, and model result analysis into the final project report.

---

1 Background and Motivation

Credit card fraud detection is a highly relevant machine learning topic because financial transactions occur at very large scale, while fraudulent events are rare, costly, and constantly evolving. Recent review studies show that machine learning and deep learning have become central approaches in this area, since traditional rule-based systems are often too rigid to capture new fraud patterns or adapt quickly to changing attack behaviors [3, 2].

From an application perspective, the task is challenging for at least three reasons. First, fraud data are extremely imbalanced: only a very small fraction of transactions are fraudulent. Second, the behavior of fraudsters changes over time, which creates concept drift and reduces the stability of static models. Third, real systems must balance fraud recall with false alarms, because blocking legitimate customers also creates business loss and poor user experience [5, 6].

For a machine learning project, this topic is a good choice because it combines practical importance with clear technical challenges. It not only allows the group to study machine learning methods in depth, but also provides an opportunity to discuss how models should be evaluated under class imbalance. In this setting, metrics such as precision, recall, F1-score, and ROC-AUC are usually more informative than accuracy alone, because a model can easily obtain very high accuracy by simply predicting most transactions as non-fraudulent [3, 2].

Our project focuses on credit card fraud detection as a binary classification problem. The goal is to learn from historical transaction records and predict whether a new transaction is legitimate or fraudulent. This is a meaningful end-to-end ML application because it involves data understanding, preprocessing, model training, evaluation, and interpretation under realistic constraints.

2 Problem Statement

Given a historical set of labeled credit card transactions, the objective is to build a classification model that can identify fraudulent transactions as accurately as possible while keeping false positives under control. In practical terms, the model should detect rare fraud cases from a highly imbalanced dataset and generalize well to unseen transactions.

Formally, given a dataset

$$D = \{(x_i, y_i)\}_{i=1}^{N},$$

where $x_i \in \mathbb{R}^d$ represents the feature vector of a transaction and $y_i \in \{0, 1\}$ indicates whether the transaction is fraudulent, the goal is to learn a function

$$f : \mathbb{R}^d \to \{0, 1\}$$

such that

$$\hat{y}_i = f(x_i),$$

which can accurately predict fraud cases while minimizing false positives.

3 Dataset

We use the well-known **Credit Card Fraud Detection** dataset from MLG-ULB, hosted on Kaggle [4, 1] (MLG-ULB, n.d.; Dal Pozzolo et al., 2015). It contains anonymized credit card transactions made by European cardholders over a two-day period in September 2013. The dataset is widely used as a benchmark for fraud detection and imbalanced learning.

Dataset Summary

Table 1: Summary of the dataset used in this project.

Item	Description
Dataset name	Credit Card Fraud Detection
Source/URL	Kaggle (MLG-ULB)
Task type	Binary classification
Number of samples	284,807
Fraudulent samples	492
Legitimate samples	284,315
Fraud ratio	0.172%
Number of columns	31
Features	30 numeric variables: Time, Amount, V1 to V28
Target label	Class (0 = legitimate, 1 = fraud)
Data characteristics	Highly imbalanced, anonymized, all-numeric dataset suitable for supervised ML experiments

In this dataset, most variables are anonymized because of confidentiality restrictions. Features V1 to V28 are principal components obtained through PCA transformation, so their original business meanings are not disclosed. The only features that were not transformed by PCA are Time and Amount. Time represents the number of seconds elapsed between each transaction and the first transaction in the dataset, while Amount records the transaction value. The label variable Class indicates whether a transaction is fraudulent (1) or not (0) (MLG-ULB, n.d.).

This dataset is especially useful for coursework because it is publicly available, clean, and already labeled, yet still challenging enough to demonstrate core ML issues such as preprocessing, class imbalance, model comparison, and performance evaluation. At the same time, its limitations should be acknowledged: the original feature meanings are hidden, the observation window is short, and the dataset may not fully reflect recent fraud patterns or richer behavioral information available in real banking systems.

Train/Test Split from Our Code

Our training script performs a stratified 80/20 split with `random_state=42`, producing:

- **Train size:** 226,980 samples
- **Test size:** 56,746 samples

All preprocessing components (scalers, SMOTE) are fit strictly on the training set to avoid leakage.

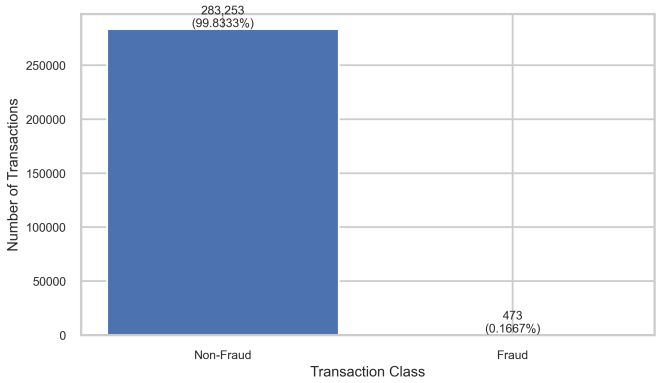


Figure 1: Class Distribution of Fraudulent vs. Non-Fraudulent Transactions

4 Exploratory Data Analysis (EDA)

4.1 Class Distribution

The dataset is highly imbalanced. Non-fraudulent transactions account for approximately 99.83% of all records, while fraudulent transactions represent only about 0.17% (Fig. 1). This extreme imbalance indicates that accuracy alone is not an appropriate evaluation metric, as a model can achieve high accuracy by predicting most transactions as non-fraud.

4.2 Transaction Amount Distribution

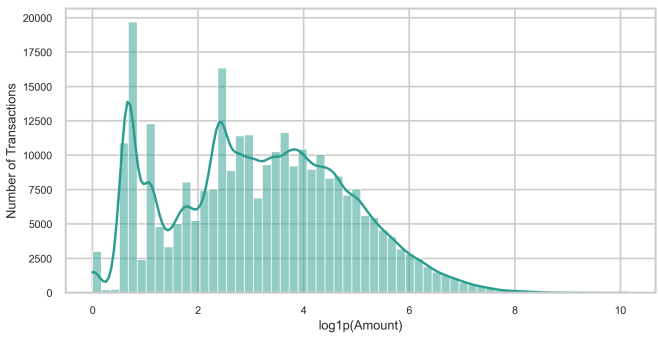


Figure 2: Distribution of Transaction Amounts

The distribution of transaction amounts is highly right-skewed, with most transactions concentrated at low values and a small number of very large transactions (Fig. 2). This suggests the presence of a long-tailed distribution, which may affect model performance. Feature scaling or transformation may be necessary in later modeling stages.

4.3 Transaction Amount by Class

The boxplot illustrates differences in transaction amounts between fraudulent and non-fraudulent transactions. Fraudulent transactions tend to have a lower median amount and a wider spread compared to normal transactions (Fig. 3). However, there remains considerable overlap between the two groups, indicating that transaction amount alone is insufficient to distinguish

fraud.

Figure 3: Transaction Amount Distribution by Class

4.4 Fraud Rate by Hour of Day

The fraud rate varies across different hours of the day. Higher fraud rates are observed during certain early hours, suggesting that fraudulent activities may follow temporal patterns (Fig. 4). This indicates that time-related features could provide useful information for fraud detection.

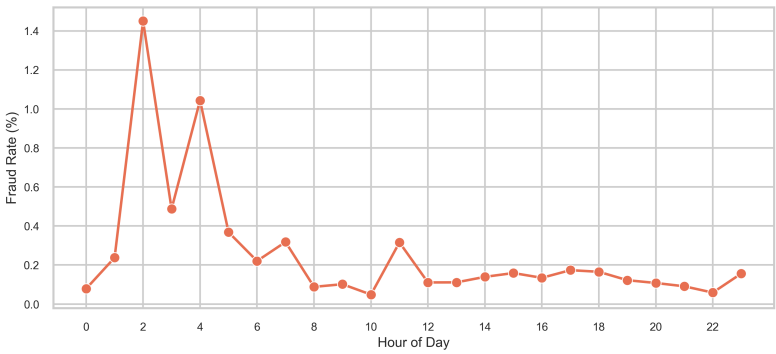


Figure 4: Fraud Rate Across Hours of the Day

4.5 Summary of Findings

Based on the EDA, the following key observations can be made:

- The dataset is extremely imbalanced, which requires careful handling during model training and evaluation.
- Transaction amounts are highly skewed; preprocessing such as scaling or transformation may be necessary.
- Fraudulent transactions show different distribution patterns from normal transactions, but no single feature is sufficient for clean separation.
- Temporal patterns suggest that fraud is not uniformly distributed over time and that time-based features may enhance model performance.

5 Preprocessing

We apply minimal, leakage-safe preprocessing consistent with the code pipeline:

1. **De-duplication:** remove exact duplicate rows.
2. **Missing values:** none present (asserted in code).
3. **Scaling:** fit `StandardScaler` on the training split for `Time` and `Amount`; create `Time_scaled`, `Amount_scaled`; then *drop* original `Time`, `Amount`. PCA features `V1`–`V28` are left as is.
4. **Imbalance handling:** apply `SMOTE` *only on the training split* (after the 80/20 split). The resampled training size becomes **453,204**. No resampling is applied to the test set.

All fitting steps (scalers, SMOTE) are performed strictly on the training data and persisted for reuse.

Table 2: Preprocessing pipeline summary (as implemented).

Step	Description
Deduplication	`drop_duplicates()` on full data before split
Split	Stratified 80/20 with `random_state = 42`
Scaling	`StandardScaler` on `Time`, `Amount`; produce `Time_scaled`, `Amount_scaled`; drop originals
Imbalance	`SMOTE` on training split only; resampled train size = 453,204
Persisted Artifacts	`scaler_time.pkl`, `scaler_amount.pkl` under `results/`

Note. Exact steps and artifacts per the training script.

6 Method

We compare two standard classifiers implemented exactly as in the training script:

- **Logistic Regression** (`solver=lbfgs`, `max_iter=1000`, `random_state=42`).
- **Random Forest** (`n_estimators=100`, `class_weight='balanced'`, `random_state=42`, `n_jobs=-1`).

Both models are trained on the *SMOTE-resampled* training set. We evaluate on the original, imbalanced test set using probability scores for ROC–AUC/AUPRC and class labels (default threshold 0.5) for Precision/Recall/F1.

7 Validation

We perform **Stratified 5-Fold Cross-Validation** on the *SMOTE-resampled training set* with `random_state=42`. For each fold we report ROC–AUC and AUPRC using predicted probabilities. After CV, we retrain each model on the full resampled training set and finally evaluate on the untouched test split. Threshold for class labels is the default 0.5; threshold tuning is left for future work.

Table 3: Cross-validation performance (mean ± SD on resampled training).

Model	ROC–AUC (CV)	AUPRC (CV)
Logistic Regression	0.9909 ± 0.0002	0.9920 ± 0.0002
Random Forest	1.0000 ± 0.0000	1.0000 ± 0.0000

8 Evaluation

After retraining on the full resampled training set, we compute on the original test split: (i) **ROC–AUC** and **AUPRC** from predicted probabilities; (ii) **Accuracy**, **Precision**, **Recall**, and **F1** from class labels produced at the default threshold 0.5 (`predict` and `predict_proba[:,1]`). Confusion matrices are computed from these class labels and exported as CSV for plotting. All artifacts (metrics JSON, predictions, confusion matrices) are saved to `results/`.

Table 4: Test-set metrics at the selected decision threshold. (This is the real table from results.zip)

Model	Accuracy	Precision	Recall	F1	ROC–AUC	AUPRC
Logistic Regression	0.9737	0.0531	0.8737	0.1002	0.9619	0.6768
Random Forest	0.9995	0.9114	0.7579	0.8276	0.9609	0.8123

Note. Scores computed on the test split (size = 56,746; train size = 226,980; SMOTE-augmented training samples = 453,204).

Table 5: Confusion matrices on the test set (counts). (This is the real table from results.zip)

(a) Logistic Regression

	Pred_Legit	Pred_Fraud
Actual_Legit	55172	1479
Actual_Fraud	12	83

(b) Random Forest

	Pred_Legit	Pred_Fraud
Actual_Legit	56644	7
Actual_Fraud	23	72

Note. Rows are actual classes; columns are predicted classes.

8.1 Interpretation

1. Random Forest provides superior practical performance

Random Forest achieves much stronger real-world performance than Logistic Regression. While Logistic Regression attains high recall (0.8737), it suffers from extremely low precision (0.0531), producing many false positives. This leads to a very low F1-score (0.1002). In contrast, Random Forest achieves both high precision (0.9114) and competitive recall (0.7579), resulting in a substantially higher F1-score of 0.8276.

This difference indicates that Logistic Regression over-predicts fraud and produces excessive false positives, whereas Random Forest achieves a more reliable and balanced decision boundary. This behavior is clearly reflected in the metric comparison (Fig. 5) and the confusion matrices (Fig. 6)

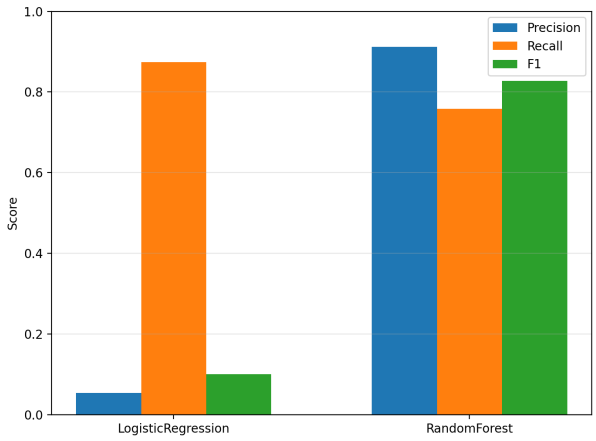
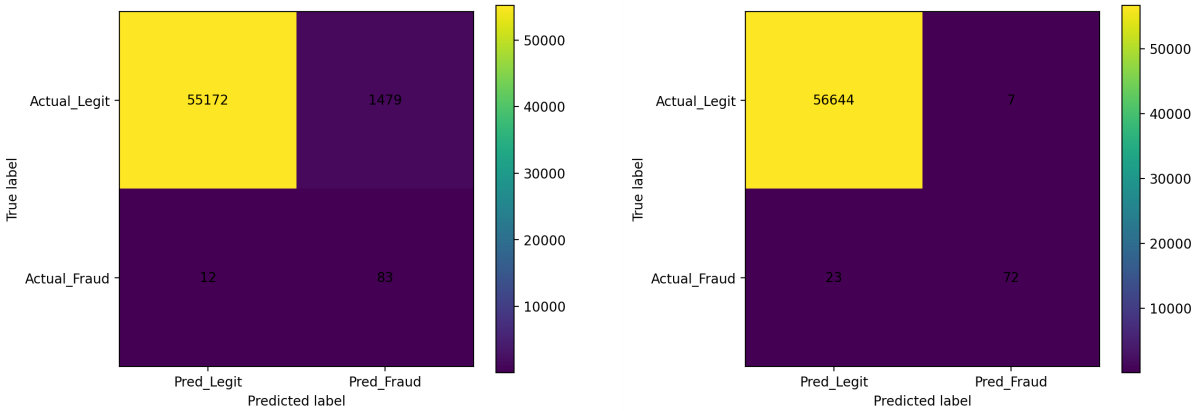


Figure 5: Random Forest VS Logistic Regression



(a) Confusion Matrix for Logistic Regression **(b)** Confusion Matrix for Random Forest

Figure 6: Confusion Matrix

2. ROC–AUC is misleading under extreme class imbalance

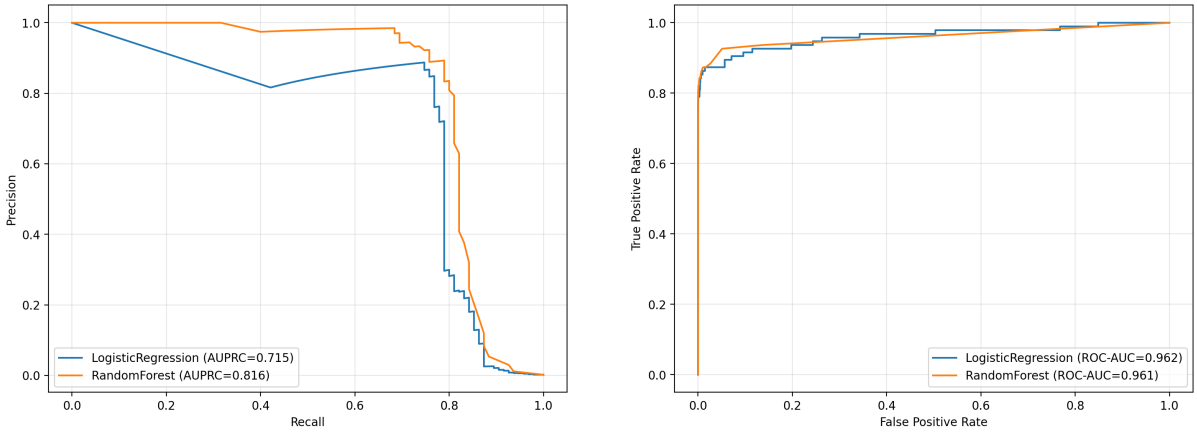
Although both models achieve similar ROC-AUC scores (≈0.96), their real-world performance differs substantially. The Precision–Recall curves (Fig. 7a) show that Random Forest consistently outperforms Logistic Regression across recall levels, achieving a higher AUPRC (0.816 vs. 0.715).

This demonstrates that ROC-AUC (Fig. 7b) fails to capture meaningful differences in highly imbalanced settings, while AUPRC provides a more faithful evaluation of fraud detection performance.

3. Cross-validation results are overly optimistic

A large discrepancy is observed between cross-validation and test-set performance (Fig. 8), with both models achieving near-perfect validation scores but significantly lower test performance.

This suggests that the validation process is overly optimistic, likely due to applying SMOTE before fold splitting, which introduces information leakage across folds. Therefore, the test set provides a more reliable estimate of model performance.



(a) Precision-Recall Curves **(b)** ROC Curves

Figure 7: Curve metric

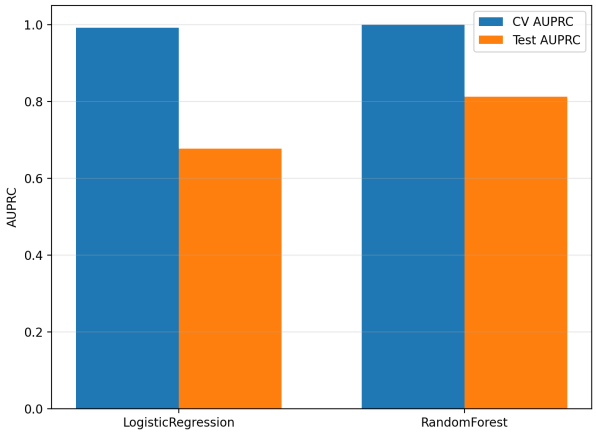


Figure 8: Cross-Validation VS Test AUPRC

4. Key insights

- **Model choice matters:** Random Forest is clearly superior in balancing precision and recall.
- **Metric choice matters:** ROC-AUC can be misleading in imbalanced settings; AUPRC provides more meaningful insights.
- **Evaluation design matters:** Improper use of resampling techniques can lead to overly optimistic validation results.

These findings highlight the importance of choosing appropriate models, metrics, and evaluation pipelines when dealing with fraud detection.

9 Limitations and Future Work

- **Threshold sensitivity:** current class labels use the default 0.5 cutoff; operating points can be tuned (e.g., maximize F1 or constrain recall) to better reflect fraud costs.

- **Resampling bias:** CV was run on the SMOTE-resampled training set; while appropriate for learning, we should also assess calibration and precision at low FPR on the original distribution.
- **Calibration:** add Platt scaling or isotonic regression on a validation set to improve probability quality for decisioning.
- **Model scope:** evaluate gradient boosting baselines (e.g., XGBoost/LightGBM) and cost-sensitive learning; consider PR-AUC optimization and threshold curves.
- **Temporal generalization:** `Time` is scaled but not used for time-aware splitting; future work could simulate deployment drift with chronological splits.

References

[1] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 159–166. IEEE, 2015.

[2] Islam Y. Hafez, Ahmed Y. Hafez, Ahmed Saleh, A. A. Abd El-Mageed, and A. A. Abohany. A systematic review of ai-enhanced techniques in credit card fraud detection. *Journal of Big Data*, 12:Article 6, 2025.

[3] E. A. L. Marazqah Btoush, Xiaoqing Zhou, Ramaswamy Gururajan, K. C. Chan, Robert Genrich, and Prakash Sankaran. A systematic review of literature on credit card cyber fraud detection using machine and deep learning. *PeerJ Computer Science*, 9:e1278, 2023.

[4] MLG-ULB. Credit card fraud detection. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, n.d. Retrieved March 20, 2026.

[5] T. Museba and K. Vanhoof. An adaptive heterogeneous ensemble learning model for credit card fraud detection. *Advances in Science, Technology and Engineering Systems Journal*, 9(3):1–11, 2024.

[6] E. Oztemel and M. Isik. A systematic review of intelligent systems and analytic applications in credit card fraud detection. *Applied Sciences*, 15(3):1356, 2025.

A Saved Artifacts and Reuse

For reproducibility and downstream visualization:

- Trained models: `results/LogisticRegression_model.pkl`, `results/RandomForest_model.pkl`.
- Scalers: `results/scaler_time.pkl`, `results/scaler_amount.pkl`.
- Metrics: `results/metrics.json` (contains CV means/SDs and all test metrics, plus split sizes).
- Test predictions: `results/test_predictions.csv` with true labels, predictions, and probability scores.
- Confusion matrices: `results/confusion_matrix_.csv`.

These files are produced automatically by the training script and enable consistent figures across group members.

B Reproducibility Details

- **Random seed:** `random_state = 42` for NumPy, split, and CV.
- **Split:** stratified 80/20; train size = 226,980; test size = 56,746.
- **Imbalance:** `SMOTE` on training split only; resampled training size = 453,204.
- **Scalers:** `StandardScaler` on `Time`, `Amount`; serialized pickles for reuse.
- **Models:** Logistic Regression (`lbfgs`, `max_iter=1000`); Random Forest (`n_estimators=100`, `class_weight='balanced'`, `n_jobs=-1`).
- **Cross-validation:** Stratified 5-Fold on the SMOTE training set; metrics reported: ROC–AUC and AUPRC (means and SDs).
- **Artifacts:** models, scalers, metrics JSON, predictions CSV, confusion matrices CSV under `results/`.

All items are implemented or emitted by the training script.